NATIONAL UNIVERSITY OF MODERN LANGUAGES

VISUAL PROGRAMMING Lab Reports

NAME: Abdul Rafay

Roll No: FL23791

PROGRAM: BSSE (6th Semester)

COURSE: Visual Programming

SUBMITTED TO: Sir M Irtaza

DATE: 5/4/2026 **DAY: Tuesday**

Table of Contents

Lab 1: Design and Implementation of a Ticket Booking System Using Windows Forms (.NET Framework)

1. Objective

The objective of this lab is to design and develop a desktop-based Ticket Booking System using Windows Forms in C#. The system allows the user to:

- Sign up with their details
- Select travel cities and date
- Confirm ticket booking
- Display booking details on the final screen

2. Introduction

This project is a Windows Forms desktop application developed using C# and the .NET Framework. The system simulates a basic ticket booking process and consists of three forms:

- Signup Form
- Booking Form
- Confirmation Form

The application demonstrates form navigation, data passing between forms, input validation, and basic GUI design — laying the foundation for the multi-form, event-driven programming style used throughout the rest of this manual.

3. Software and Tools Used

- Microsoft Visual Studio
- C# Programming Language
- .NET Framework
- Windows Forms

4. Form Details

Form1 — Signup Form

Purpose: Collect user information (Name, Email, Password).

Controls Used: Labels, TextBoxes, Button (Signup).

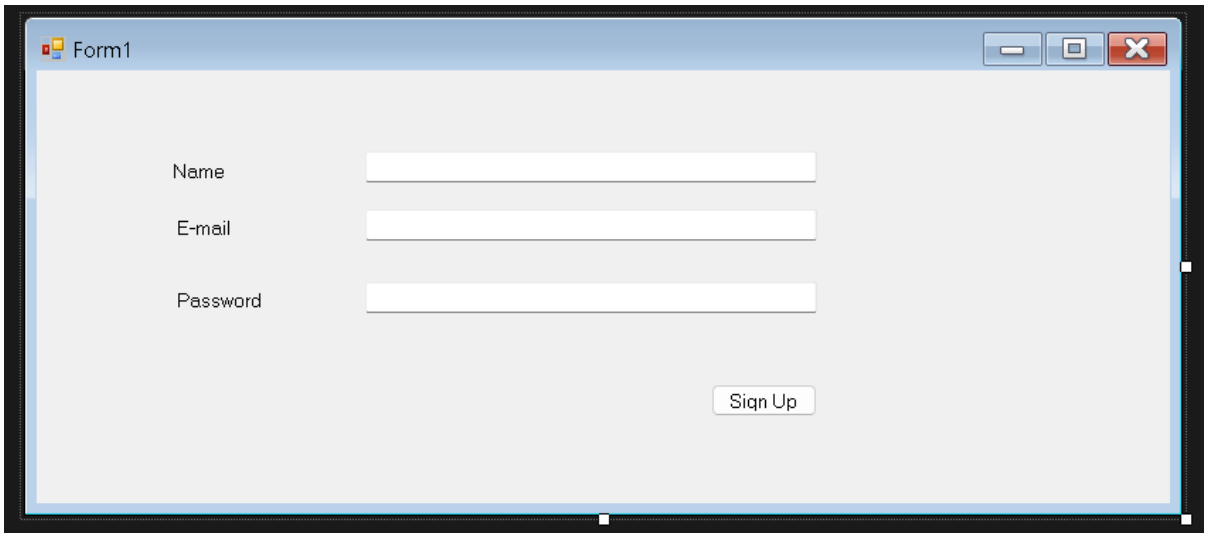


Figure 1.1 — Signup Form design

Form2 — Booking Form

Purpose: Allow the user to select the departure city, destination city, and travel date.

Controls Used: Labels, ComboBoxes, DateTimePicker, Button (Book).

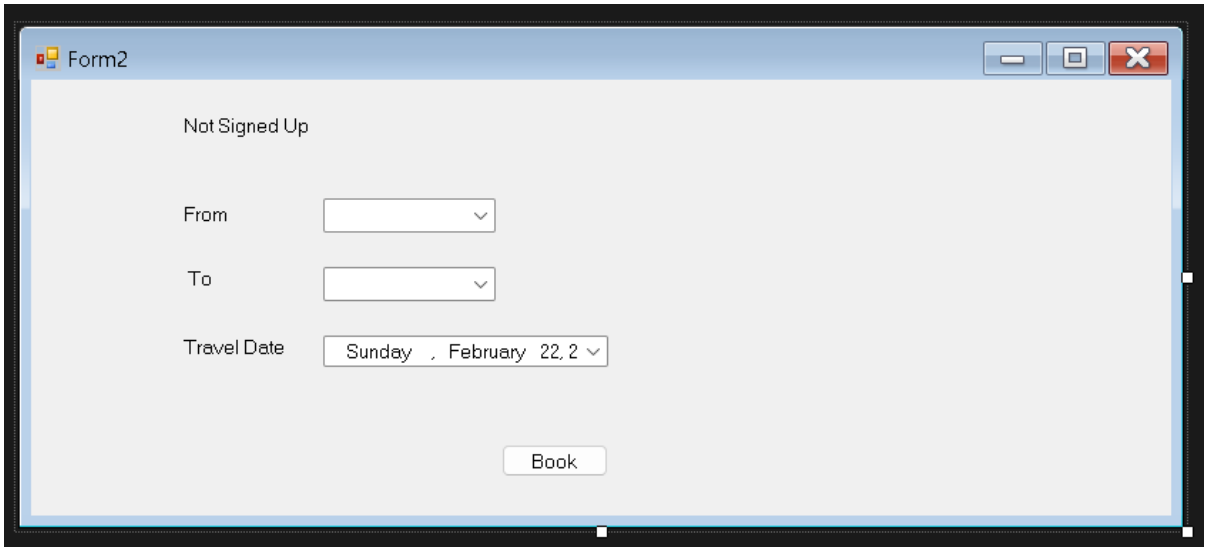


Figure 1.2 — Booking Form design

Form3 — Confirmation Form

Purpose: Display the booking confirmation and ticket details.

Controls Used: Label (for displaying booking details).

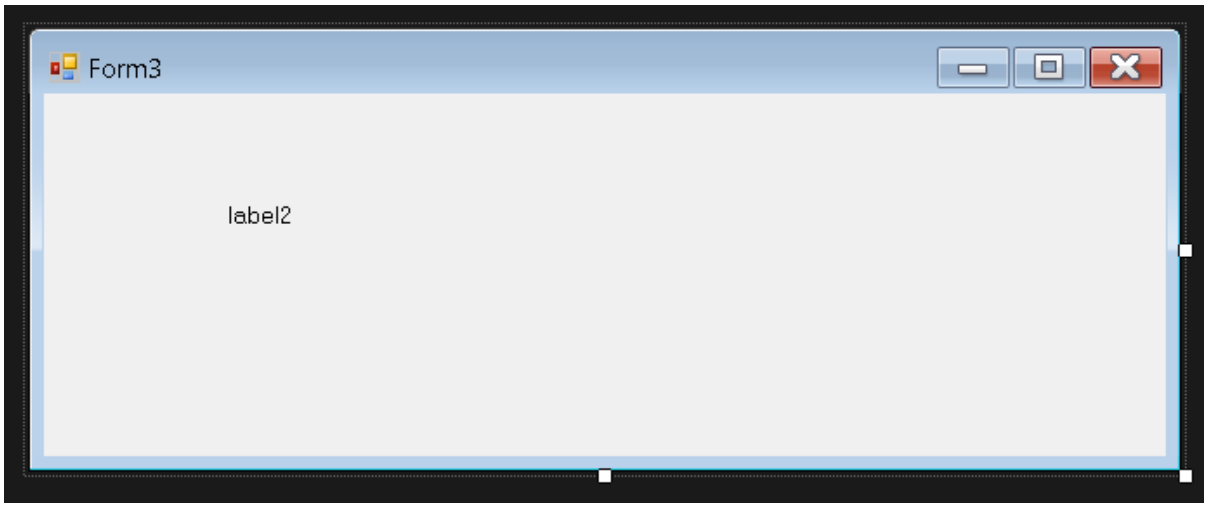


Figure 1.3 — Confirmation Form design

5. Output / Execution

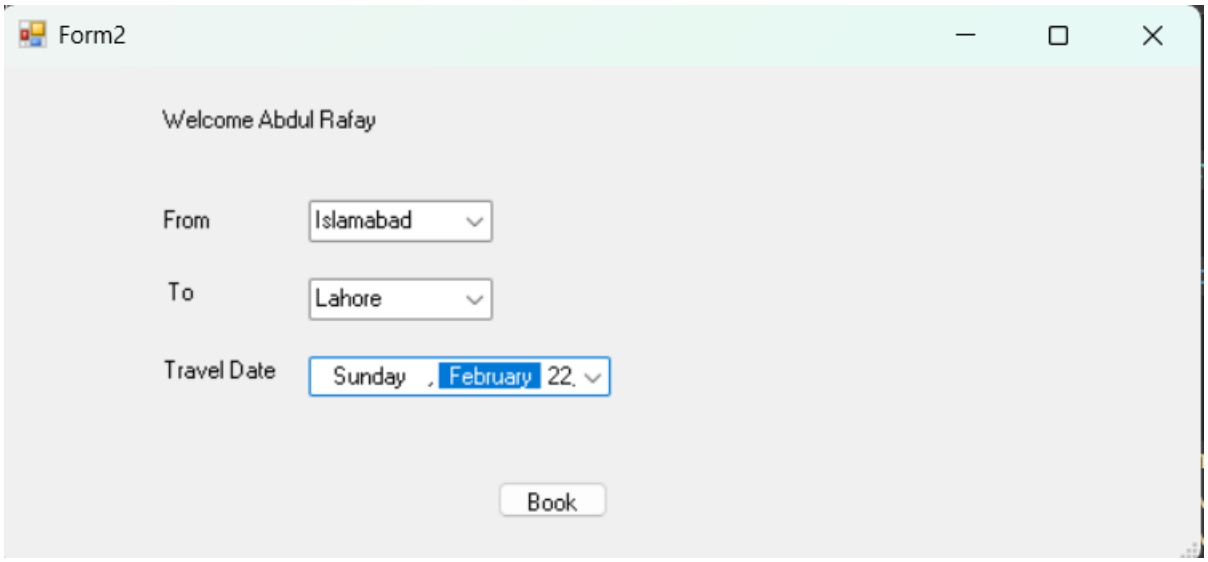


Figure 1.4 — Signup Form at runtime

Figure 1.5 — Booking Form at runtime

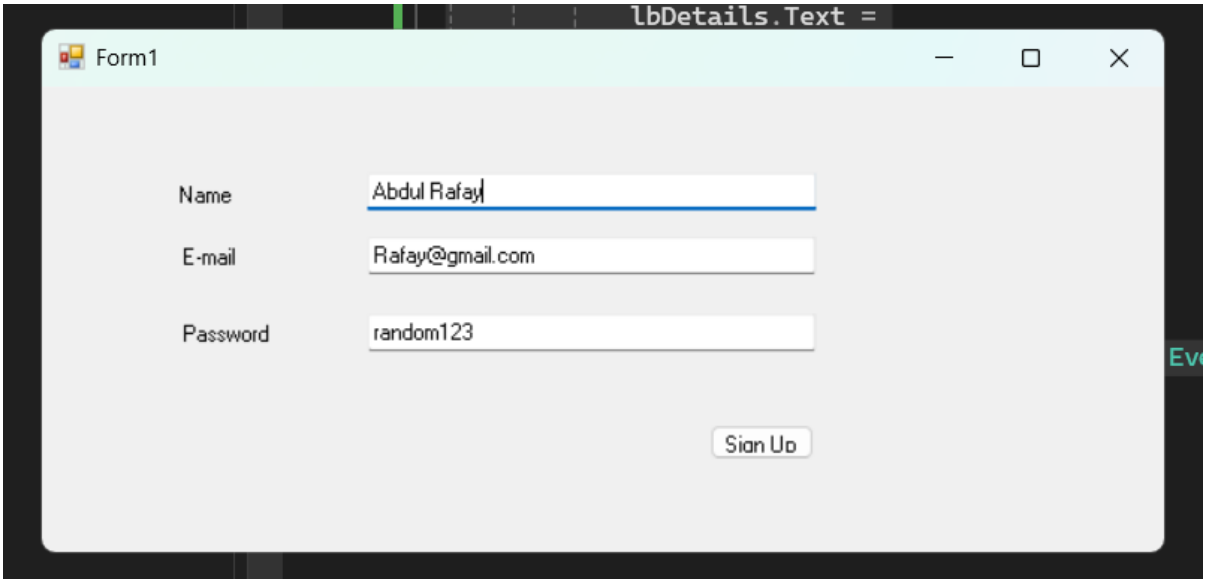


Figure 1.6 — Confirmation screen showing booking details

6. Conclusion

This lab demonstrated the fundamentals of building a multi-form Windows Forms application: passing data between forms through constructors, validating user input before proceeding, and using simple controls (TextBoxes, ComboBoxes, a DateTimePicker, and Labels) to build a working end-to-end user flow. These fundamentals of form navigation and event-driven programming carry forward into the remainder of this lab series.

Lab 2: Database Connectivity

Objective

To design a relational database and establish connectivity between a C# Windows Forms application and Microsoft SQL Server using ADO.NET.

Theory

ADO.NET is a data access technology in the .NET Framework that allows applications to communicate with relational databases such as SQL Server. It acts as a bridge between the front-end application and the back-end database, enabling operations like reading, inserting, updating, and deleting records.

Key ADO.NET Components

- **Connection Object (SqlConnection):** Establishes a physical connection to the database using a connection string containing the server name, database name, and authentication details.
- **Command Object (SqlCommand):** Represents an SQL statement or stored procedure to be executed against the database.
- **DataReader (SqlDataReader):** Provides a forward-only, read-only stream of data — efficient for reading large result sets.
- **DataAdapter (SqlDataAdapter):** Acts as a bridge between a DataSet/DataTable and the database, filling and updating data in memory.
- **DataSet / DataTable:** An in-memory, disconnected representation of data that can hold one or more tables.

Connection String

A connection string is a string containing the parameters needed to establish a database connection — server address, database name, and security credentials (Windows Authentication or SQL Server Authentication).

Database Design

The core entities designed for this project are:

- **Books** (BookID, Title, Author, ISBN, Category, Quantity, AvailableCopies)
- **Students / Members** (StudentID, Name, Department, ContactNo, Email)
- **Transactions / IssueRecords** (TransactionID, BookID, StudentID, IssueDate, DueDate, ReturnDate, Status)
- **Staff / Admin** (StaffID, Username, Password, Role)

Normalization is applied (up to 3NF) to avoid redundancy — for example, keeping book details separate from transaction details rather than repeating book information in every issue record.

Tools Required

- Visual Studio (2019 / 2022)

- Microsoft SQL Server and SQL Server Management Studio (SSMS)
- .NET Framework — Windows Forms Application

Procedure

1. Create the database in SSMS with the tables described above.
2. Create a Windows Forms project in Visual Studio.
3. Add a connection string to App.config, or define it directly in code.
4. Use SqlConnection to open a connection and test connectivity with a simple query (e.g., fetch the total book count).

Connection String / Connectivity Code

App.config | signup.csproj | Form2.Designer.cs | Form2.cs | Employees.cs | Form1.cs

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
    <connectionStrings>
        <add name="DBConnection"
             connectionString="Data Source=SE-DEVELOPLAB37\SQLEXPRESS;Initial Catalog=OpenEndedLab;Integrated Security=True"
             providerName="System.Data.SqlClient"/>
    </connectionStrings>

    <startup>
        <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.7.2" />
    </startup>
</configuration>
```

Screenshots

Conclusion

This lab establishes the foundation of the project by linking the application layer to persistent storage, which subsequent labs build upon.

Lab 3: CRUD Operations using SQL Queries

Objective

To implement Create, Read, Update, and Delete (CRUD) operations for the Books and Students modules using raw SQL queries through ADO.NET.

Theory

CRUD stands for Create, Read, Update, Delete — the four basic operations performed on persistent data storage, forming the backbone of nearly all data-driven applications.

Operation	SQL Statement	Purpose
Create	INSERT INTO	Add a new record (new book / student)
Read	SELECT	Retrieve or search records
Update	UPDATE	Modify an existing record (e.g., available copies)
Delete	DELETE	Remove a record

Parameterized Queries

Instead of concatenating user input directly into SQL strings — which is vulnerable to SQL Injection — parameterized queries use placeholders (@ParamName) bound to SqlParameter objects. This is both a security best practice and improves query-plan reuse.

Search Functionality

Search is typically implemented using the LIKE operator with wildcards (%) to allow partial matches, e.g., searching book titles containing a keyword.

Validation

Before executing any CRUD operation, input validation (empty fields, duplicate ISBN, numeric-only quantity, etc.) should be enforced at the application layer to maintain data integrity.

ExecuteNonQuery vs ExecuteReader vs ExecuteScalar

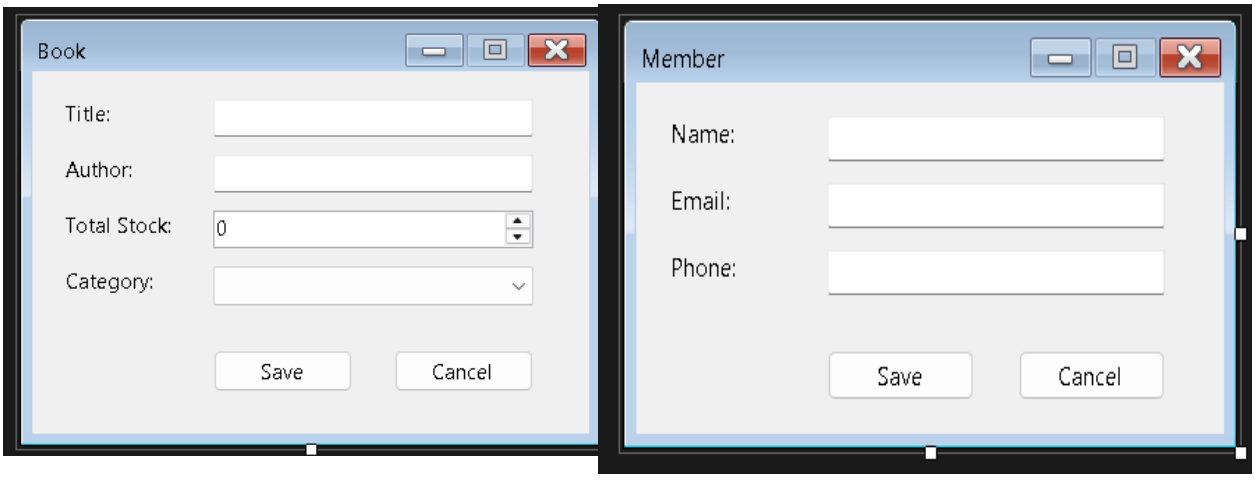
- **ExecuteNonQuery():** used for INSERT, UPDATE, DELETE (returns the number of rows affected).
- **ExecuteReader():** used for SELECT statements returning multiple rows.
- **ExecuteScalar():** used for SELECT statements returning a single value (e.g., COUNT).

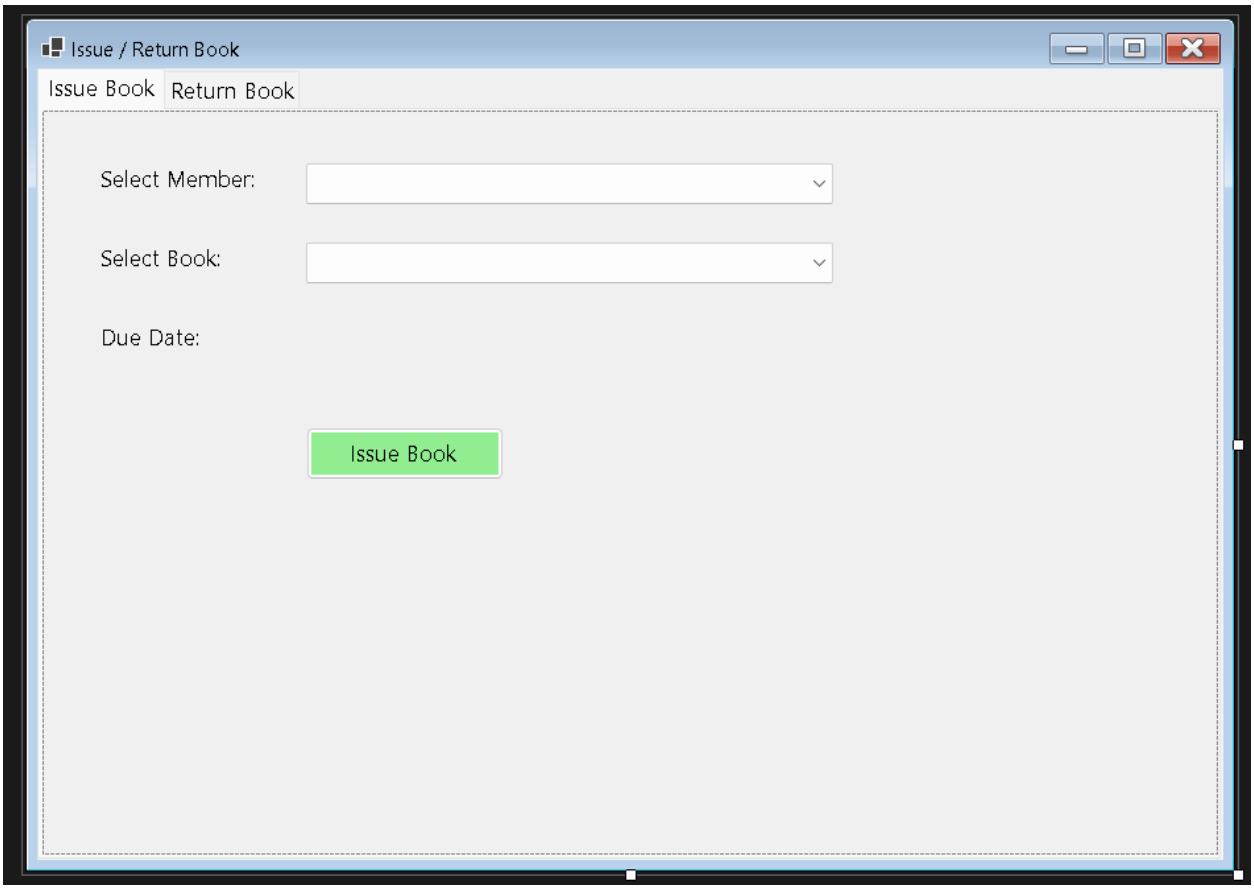
Procedure

5. Design forms for Add Book, Update Book, Delete Book, and Search Book.
6. Repeat the same set of operations for the Student module.
7. Implement each operation using parameterized SqlCommand objects.
8. Handle exceptions using try-catch and provide user feedback via MessageBox.

```
// Insert
string qi = "INSERT INTO Students(Name,Department,ContactNo) VALUES(@n,@d,@c)";
SqlCommand ci = new SqlCommand(qi, con);
ci.Parameters.AddWithValue("@n", txtName.Text);
ci.Parameters.AddWithValue("@d", txtDept.Text);
ci.Parameters.AddWithValue("@c", txtContact.Text);
con.Open(); ci.ExecuteNonQuery(); con.Close();

// Update
string qu = "UPDATE Students SET Name=@n,Department=@d,ContactNo=@c WHERE StudentID=@id";
SqlCommand cu = new SqlCommand(qu, con);
cu.Parameters.AddWithValue("@n", txtName.Text);
cu.Parameters.AddWithValue("@d", txtDept.Text);
cu.Parameters.AddWithValue("@c", txtContact.Text);
cu.Parameters.AddWithValue("@id", txtStudentID.Text);
con.Open(); cu.ExecuteNonQuery(); con.Close();

// Delete
string qd = "DELETE FROM Students WHERE StudentID=@id";
SqlCommand cd = new SqlCommand(qd, con);
cd.Parameters.AddWithValue("@id", txtStudentID.Text);
con.Open(); cd.ExecuteNonQuery(); con.Close();
```

Screenshots





Conclusion

CRUD operations form the operational core of the system, allowing a librarian to manage book and student records dynamically.

Lab 4: Displaying Records in a DataGridView

Objective

To display and manipulate Book and Student records from the database in a DataGridView control within the Windows Forms application.

Theory

DataGridView is a Windows Forms control used to display tabular data in a grid format, similar to a spreadsheet. It supports data binding, sorting, filtering, editing, and custom formatting.

Binding Methods

9. **Manual Binding:** Populate the DataGridView row-by-row in a loop using SqlDataReader.
10. **DataSource Binding:** Assign a DataTable (filled via SqlDataAdapter.Fill()) directly to dataGridView.DataSource. This is more efficient and automatically handles column generation.

Key DataGridView Features Used

- **Column Customization:** Renaming headers, hiding unnecessary columns (like internal IDs), setting column widths.
- **Row Selection Events:** CellClick or SelectionChanged events used to populate textboxes when a row is selected — useful for edit / delete workflows.
- **Sorting:** Built-in column-header sorting.
- **Filtering:** Achieved by modifying the underlying query (e.g., a WHERE clause based on a search box) and refreshing the DataSource.
- **Refresh Pattern:** After any Insert / Update / Delete, the grid should be reloaded via a reusable LoadData() method, called after every operation.

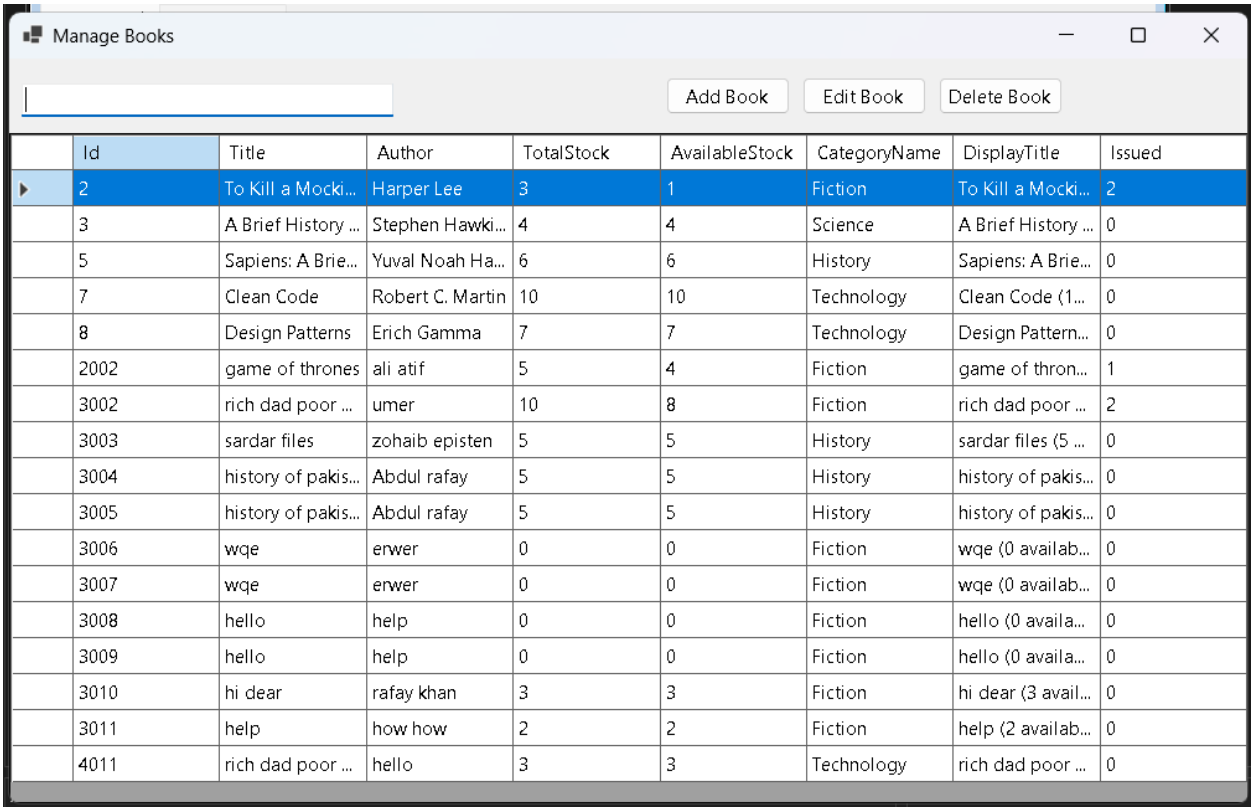
DataTable and DataAdapter Relationship

The SqlDataAdapter uses a SelectCommand to fetch data, and Fill() populates a DataTable, which acts as an in-memory cache that the DataGridView renders.

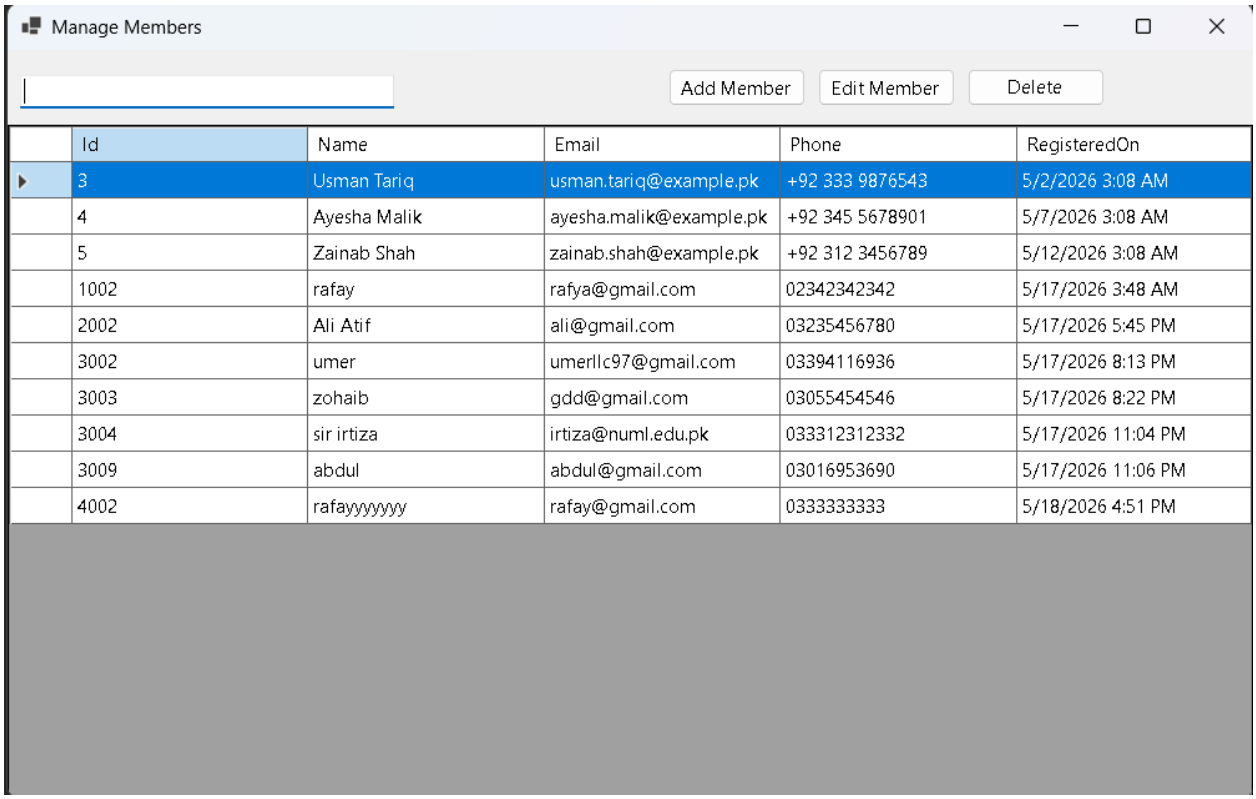
Procedure

11. Create a LoadBooks() method that fills a DataTable via SqlDataAdapter and binds it to the grid.
12. Add a search TextBox that filters grid data live or on button click.
13. On row click, populate the form fields for editing.
14. Refresh the grid after every CRUD operation implemented in Lab 3.

Screenshots



Id	Title	Author	TotalStock	AvailableStock	CategoryName	DisplayTitle	Issued
2	To Kill a Mocki...	Harper Lee	3	1	Fiction	To Kill a Mocki...	2
3	A Brief History ...	Stephen Hawki...	4	4	Science	A Brief History ...	0
5	Sapiens: A Brie...	Yuval Noah Ha...	6	6	History	Sapiens: A Brie...	0
7	Clean Code	Robert C. Martin	10	10	Technology	Clean Code (1...	0
8	Design Patterns	Erich Gamma	7	7	Technology	Design Pattern...	0
2002	game of thrones	ali atif	5	4	Fiction	game of thron...	1
3002	rich dad poor ...	umer	10	8	Fiction	rich dad poor ...	2
3003	sardar files	zohaib episten	5	5	History	sardar files (5 ...	0
3004	history of pakis...	Abdul rafay	5	5	History	history of pakis...	0
3005	history of pakis...	Abdul rafay	5	5	History	history of pakis...	0
3006	wqe	erwer	0	0	Fiction	wqe (0 availab...	0
3007	wqe	erwer	0	0	Fiction	wqe (0 availab...	0
3008	hello	help	0	0	Fiction	hello (0 availa...	0
3009	hello	help	0	0	Fiction	hello (0 availa...	0
3010	hi dear	rafay khan	3	3	Fiction	hi dear (3 avail...	0
3011	help	how how	2	2	Fiction	help (2 availab...	0
4011	rich dad poor ...	hello	3	3	Technology	rich dad poor ...	0



Id	Name	Email	Phone	RegisteredOn
3	Usman Tariq	usman.tariq@example.pk	+92 333 9876543	5/2/2026 3:08 AM
4	Ayesha Malik	ayesha.malik@example.pk	+92 345 5678901	5/7/2026 3:08 AM
5	Zainab Shah	zainab.shah@example.pk	+92 312 3456789	5/12/2026 3:08 AM
1002	rafay	rafya@gmail.com	02342342342	5/17/2026 3:48 AM
2002	Ali Atif	ali@gmail.com	03235456780	5/17/2026 5:45 PM
3002	umer	umerllc97@gmail.com	03394116936	5/17/2026 8:13 PM
3003	zohaib	gdd@gmail.com	03055454546	5/17/2026 8:22 PM
3004	sir irtiza	irtiza@numl.edu.pk	033312312332	5/17/2026 11:04 PM
3009	abdul	abdul@gmail.com	03016953690	5/17/2026 11:06 PM
4002	rafayyyyyyy	rafay@gmail.com	0333333333	5/18/2026 4:51 PM

Conclusion

The DataGridView provides a user-friendly interface for viewing and interacting with library data, bridging the raw database with usable application UI.

Lab 5: Implementing Stored Procedures

Objective

To refactor the data access layer built so far to use Stored Procedures instead of inline SQL queries for all CRUD operations.

Theory

A **Stored Procedure** is a precompiled collection of one or more SQL statements stored under a name in the database, which can be executed (called) repeatedly.

Example — sp_InsertBook (code run in sql studio)

```
CREATE PROCEDURE sp_InsertBook
    @BookName NVARCHAR(100),
    @Author NVARCHAR(100),
    @Category NVARCHAR(50),
    @Quantity INT,
    @Price DECIMAL(10,2)
AS
BEGIN
    INSERT INTO Books(BookName, Author, Category, Quantity, Price)
    VALUES(@BookName, @Author, @Category, @Quantity, @Price)
END
GO
```

Advantages of Stored Procedures

15. **Performance:** Precompiled execution plans reduce parsing and optimization overhead on repeated calls.
16. **Security:** Reduces SQL Injection risk since parameters are strongly typed and separated from logic; also allows granting execute permissions without exposing direct table access.
17. **Maintainability:** Business logic changes can be made at the database level without recompiling the application.
18. **Reusability:** The same procedure can be called from multiple applications or modules.
19. **Reduced Network Traffic:** Only the procedure name and parameters are sent over the network, not the full SQL text.

Types of Parameters

- **Input Parameters:** Pass data into the procedure (e.g., @Title).
- **Output Parameters:** Return a value from the procedure (e.g., @NewBookID OUTPUT).
- **Return Values:** Typically used for status codes (0 = success, -1 = failure).

Calling Stored Procedures from ADO.NET

The key distinction from Lab 3 is setting CommandType = CommandType.StoredProcedure instead of CommandType.Text on the SqlCommand object.

Stored Procedures vs Functions

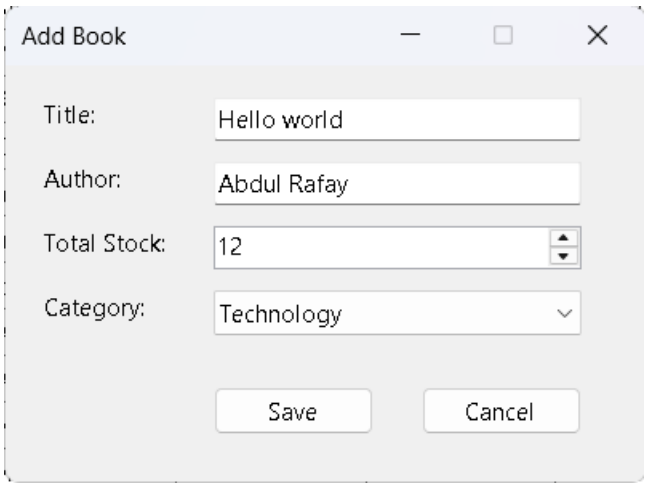
Aspect	Stored Procedure	Function
Return	Can return 0, 1, or multiple result sets	Must return a single value / table
Usage in SELECT	Cannot be used inline in SELECT	Can be used inline
DML operations	Can perform INSERT / UPDATE / DELETE	Generally restricted

Procedure

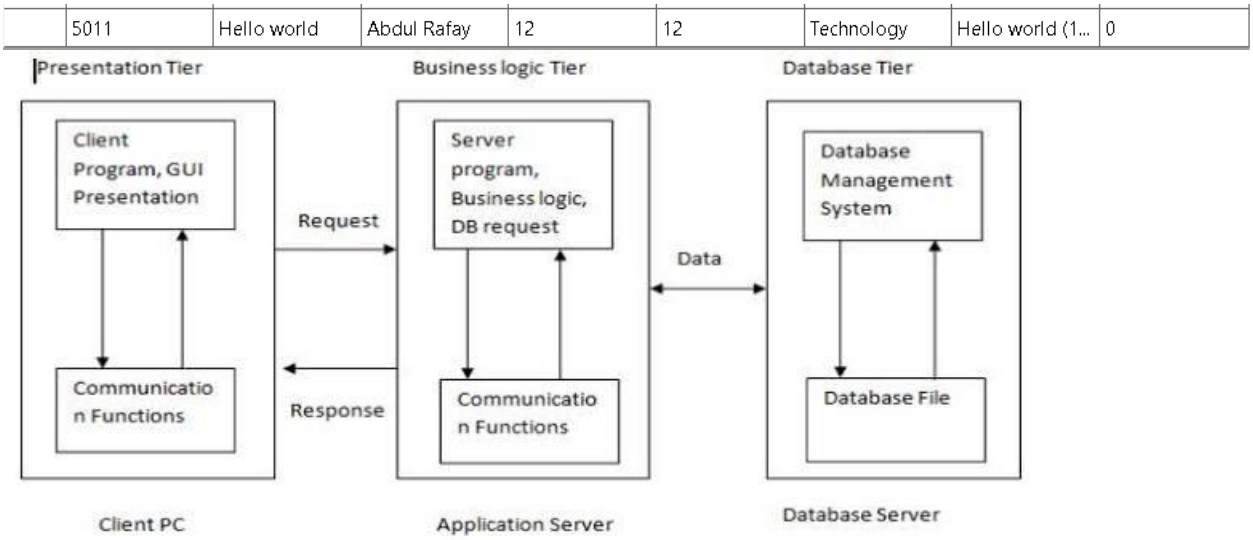
20. Write stored procedures for Insert, Update, Delete, and Select operations for Books and Students.
21. Modify the C# code to call these procedures instead of inline queries.
22. Test each operation to confirm functional equivalence with Lab 3.

Stored Procedure Scripts

```sql

CREATE PROCEDURE sp_InsertBook
    @BookName NVARCHAR(100),
    @Author NVARCHAR(100),
    @Category NVARCHAR(50),
    @Quantity INT,
    @Price DECIMAL(10,2)
AS
BEGIN
    INSERT INTO Books(BookName, Author, Category, Quantity, Price)
    VALUES(@BookName,@Author,@Category,@Quantity,@Price);
END;
GO


-- Stored Procedure : Update Book


CREATE PROCEDURE sp_UpdateBook
    @BookID INT,
    @BookName NVARCHAR(100),
    @Author NVARCHAR(100),
    @Category NVARCHAR(50),
    @Quantity INT,
    @Price DECIMAL(10,2)
AS
BEGIN
    UPDATE Books
    SET BookName=@BookName,
        Author=@Author,
        Category=@Category,
        Quantity=@Quantity,
        Price=@Price
    WHERE BookID=@BookID;
END;
GO

CREATE PROCEDURE sp_DeleteBook
    @BookID INT
AS
BEGIN
    DELETE FROM Books
    WHERE BookID=@BookID;
END;
GO


-- Stored Procedure : Display All Books


CREATE PROCEDURE sp_GetBooks
AS
BEGIN
    SELECT * FROM Books;
END;
GO


-- Stored Procedure : Insert Student


CREATE PROCEDURE sp_InsertStudent
    @StudentName NVARCHAR(100),
    @Department NVARCHAR(50),
    @Semester INT,
    @Phone NVARCHAR(20)
AS
BEGIN
    INSERT INTO Students(StudentName,Department,Semester,Phone)
    VALUES(@StudentName,@Department,@Semester,@Phone);
END;
GO
```



Book added successfully



Conclusion

Migrating to stored procedures improves the performance, security, and maintainability of the data access layer built in the previous labs.

Lab 6: Three-Tier Architecture

Objective

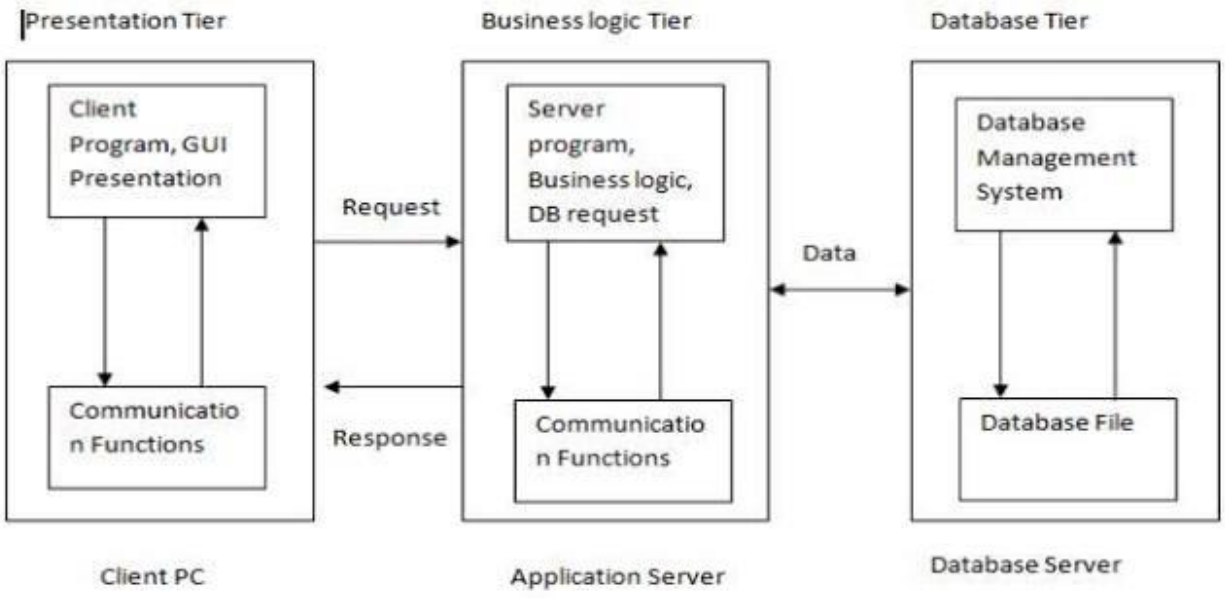
To restructure the application from a two-tier (UI + Database) design into a Three-Tier Architecture, separating the Presentation, Business Logic, and Data Access layers.

Theory

Three-Tier Architecture divides an application into three logically separated, independently maintainable layers:

23. **Presentation Layer (PL):** The UI layer — Windows Forms — handles user interaction, input collection, and displaying results. Contains no direct database code.
24. **Business Logic Layer (BLL):** Contains the application's core logic, validation rules, and workflow — deciding what should happen (e.g., a book cannot be issued if no copies are available).
25. **Data Access Layer (DAL):** Solely responsible for communicating with the database — executing queries or stored procedures and returning results. Contains no business rules.

Layered Communication Flow



Why Three-Tier?

- **Separation of Concerns:** Each layer has one responsibility, making the codebase easier to understand and maintain.
- **Reusability:** The BLL and DAL can be reused by different front-ends (e.g., later reused by the ASP.NET version in Lab 7).
- **Scalability:** Layers can be deployed on separate servers/tiers in larger systems.

- **Testability:** Business logic can be unit tested independently of the UI.
- **Maintainability:** Changes in the database schema mainly affect the DAL, not the entire application.

Implementation Approach

Separate class-library projects (or namespaces/folders) are created for the DAL, BLL, and Presentation layers. Entity classes (e.g., Book, Student) are defined as simple data-carrying objects (POCOs — Plain Old CLR Objects) used to pass data between layers. The DAL exposes methods such as InsertBook(Book b) and GetAllBooks(); the BLL calls DAL methods and adds validation (e.g., checking Quantity > 0 before insertion); and the UI calls only BLL methods — never the DAL or raw SQL directly.

Two-Tier vs Three-Tier

Aspect	**Two-Tier**	**Three-Tier**
Layers	UI + Database	UI + BLL + DAL + Database
Coupling	Tight	Loose
Maintainability	Harder as the app grows	Easier
Reusability	Low	High

Procedure

26. Create entity classes for Book, Student, and Transaction.
27. Create a DAL project/folder with classes handling raw database operations (reusing the stored procedures from Lab 5).
28. Create a BLL project/folder that calls the DAL and applies validation.
29. Update the Presentation Layer to call only BLL methods.

Entity Class Code

```
namespace LibraryManagementSystem.Entity
{
   public class Book
   {
      public int BookID { get; set; }

      public string Title { get; set; }

      public string Author { get; set; }

      public string Category { get; set; }

      public int Stock { get; set; }
```

```
    }
}
```

DAL Class Code

```
using System.Data;
using System.Data.SqlClient;
using LibraryManagementSystem.Entity;

namespace LibraryManagementSystem.DAL
{
    public class BookDAL
    {
        SqlConnection con = new SqlConnection(@"Data Source=.;Initial Catalog=LibraryDB;Integrated
Security=True");

        public void InsertBook(Book book)
        {
            SqlCommand cmd = new SqlCommand("sp_InsertBook", con);
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.AddWithValue("@Title", book.Title);
            cmd.Parameters.AddWithValue("@Author", book.Author);
            cmd.Parameters.AddWithValue("@Category", book.Category);
            cmd.Parameters.AddWithValue("@Stock", book.Stock);

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
        }

        public DataTable GetBooks()
        {
            SqlCommand cmd = new SqlCommand("sp_GetBooks", con);
            cmd.CommandType = CommandType.StoredProcedure;

            SqlDataAdapter da = new SqlDataAdapter(cmd);
            DataTable dt = new DataTable();
```

```
            da.Fill(dt);

            return dt;
        }

        public void UpdateBook(Book book)
        {
            SqlCommand cmd = new SqlCommand("sp_UpdateBook", con);
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.AddWithValue("@BookID", book.BookID);
            cmd.Parameters.AddWithValue("@Title", book.Title);
            cmd.Parameters.AddWithValue("@Author", book.Author);
            cmd.Parameters.AddWithValue("@Category", book.Category);
            cmd.Parameters.AddWithValue("@Stock", book.Stock);

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
        }

        public void DeleteBook(int id)
        {
            SqlCommand cmd = new SqlCommand("sp_DeleteBook", con);
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.AddWithValue("@BookID", id);

            con.Open();
            cmd.ExecuteNonQuery();
            con.Close();
        }
    }
}
```

BLL Class Code

```
using System.Data;
using LibraryManagementSystem.DAL;
using LibraryManagementSystem.Entity;
```

```
namespace LibraryManagementSystem.BLL
{
    public class BookBLL
    {
        BookDAL dal = new BookDAL();

        public void AddBook(Book book)
        {
            dal.InsertBook(book);
        }

        public void UpdateBook(Book book)
        {
            dal.UpdateBook(book);
        }

        public void DeleteBook(int id)
        {
            dal.DeleteBook(id);
        }

        public DataTable GetAllBooks()
        {
            return dal.GetBooks();
        }
    }
}
```

Conclusion

Three-Tier Architecture significantly improves the structure of the project, setting the stage for scalability and reuse across different front-ends, such as the ASP.NET web version built in the next lab.

Lab 7: Web Application Development using ASP.NET

Objective

To develop a web-based version of the application using ASP.NET, reusing the Business Logic and Data Access Layers built in Lab 6.

Theory

ASP.NET is a server-side web application framework developed by Microsoft for building dynamic web pages and web applications using C# or VB.NET.

Key ASP.NET Web Forms Concepts

- **.aspx Page:** Contains the markup (HTML + server controls) for the UI.
- **Code-Behind (.aspx.cs):** Contains the C# logic associated with the page, following the Page Life Cycle.
- **Server Controls:** UI elements such as GridView, TextBox, and Button that run on the server and render as HTML.
- **Page Life Cycle:** The sequence of events a page goes through — Page_Init, Page_Load, event handling (e.g., Button_Click), Page_PreRender, Page_Unload. Understanding IsPostBack is important — initialization logic in Page_Load should typically only run when !IsPostBack, to avoid re-executing on every postback.
- **State Management:** Since HTTP is stateless, ASP.NET provides ViewState (preserves control values across postbacks), Session State (stores user-specific data across a browsing session, e.g., the logged-in librarian), and Application State (shared across all users).
- **GridView Control:** The web equivalent of DataGridView, used to display Book/Student records, supporting paging, sorting, and template-based editing.
- **Postback:** A request sent from a page back to itself on the server, triggered by control events such as a button click.

Reusing the Three-Tier Layers

Since the BLL and DAL built in Lab 6 contain no UI-specific code, they can be referenced directly as a class library in the ASP.NET project — only a new Presentation Layer (Web Forms) needs to be built, demonstrating the value of the layered architecture.

Web.config

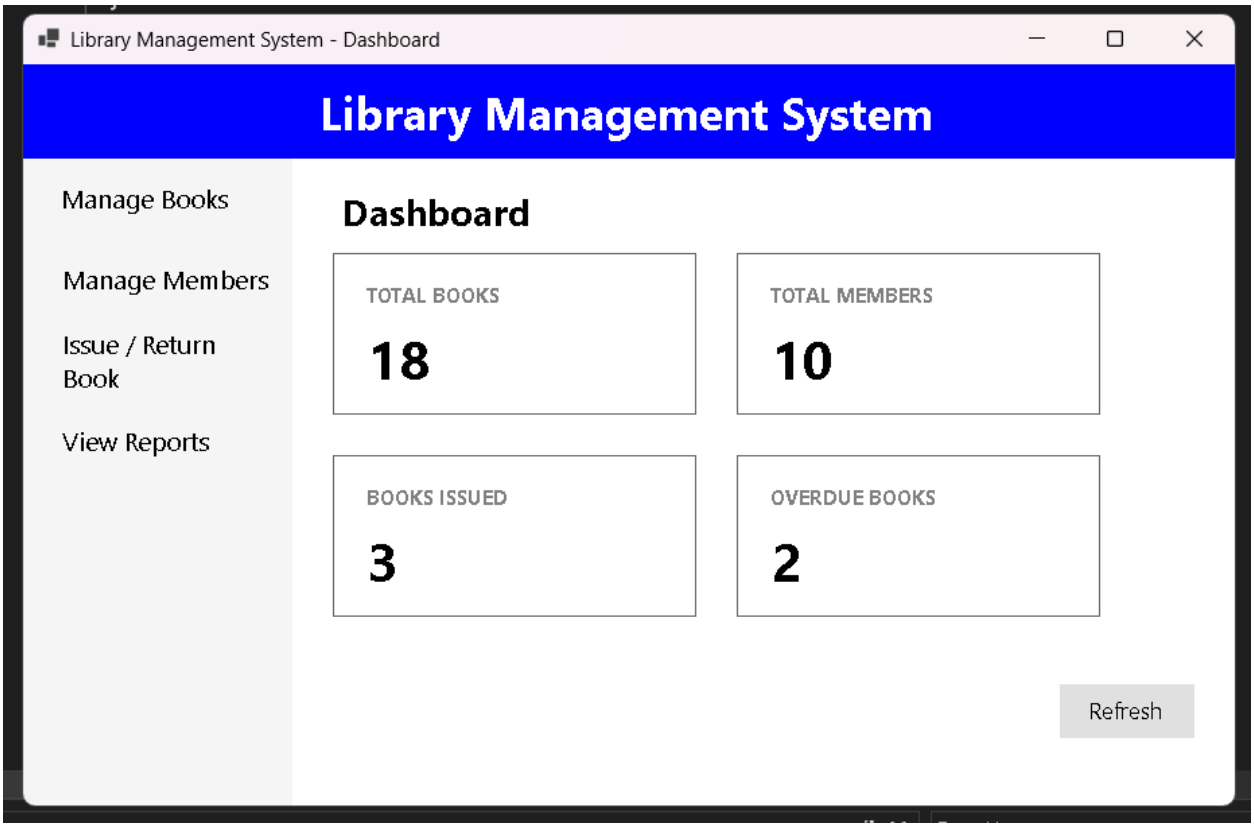
The configuration file for an ASP.NET application, storing connection strings, custom settings, and application-level configuration — analogous to App.config in a Windows Forms application.

Procedure

30. Create a new ASP.NET Web Forms project.
31. Add a project reference to the existing BLL/DAL class libraries.

32. Design web pages: BookList.aspx, AddBook.aspx, StudentList.aspx, IssueBook.aspx, Login.aspx.
33. Bind GridView controls to BLL methods.
34. Implement Session-based login for library staff.

Screenshot



Conclusion

Porting the application to ASP.NET demonstrates the reusability benefit of the layered architecture and introduces web-based data management concepts.

Lab 8: Open Ended Lab (OEL) — Employee Management System

Task

Create a program for an Employee Management System where two textboxes are used for data input, and the data is stored in the database using a DbCon class.

Form Layout

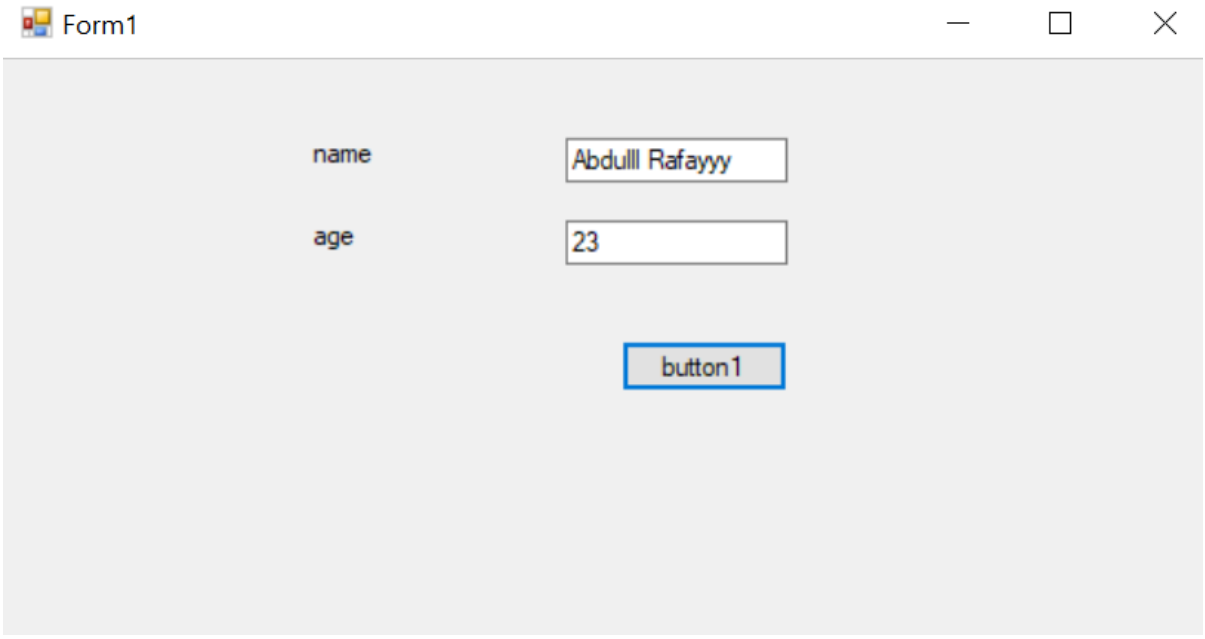


Figure 8.1 — Employee entry form (Name, Age, and a Submit button)

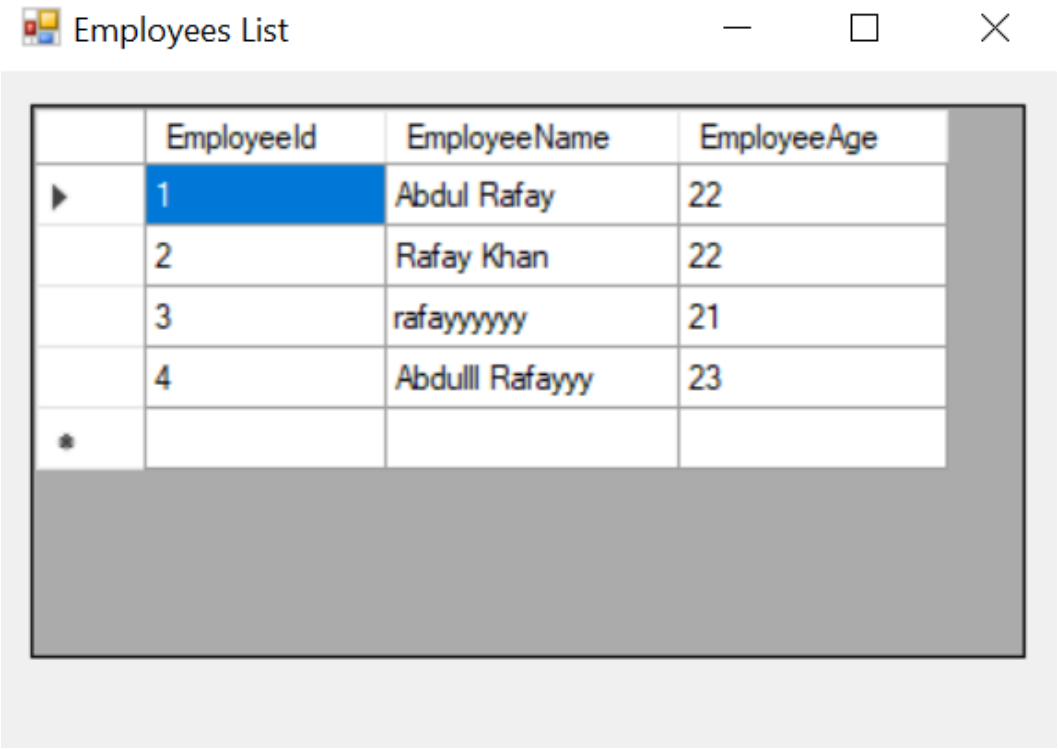


Figure 8.2 — Employees List window displaying stored records in a DataGridView

Database Creation Script (SQL Server)

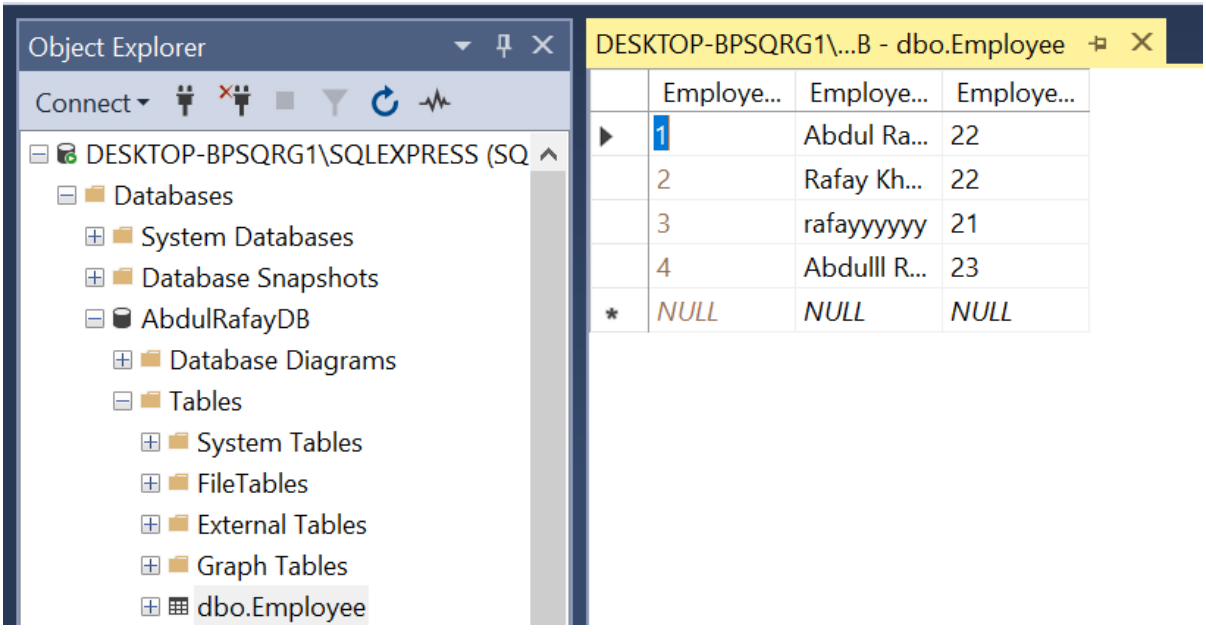


Figure 8.3 — SSMS Object Explorer showing the AbdulRafayDB database and the dbo.Employee table with sample data

Employees.cs — Data Access Class

This class defines the DbCon-style data access logic: it reads the connection string from App.config, exposes an InsertEmployee(string name, int age) method that inserts a new row using a parameterized SqlCommand, and a GetEmployees() method that fills a DataTable via SqlDataAdapter for binding to the grid.

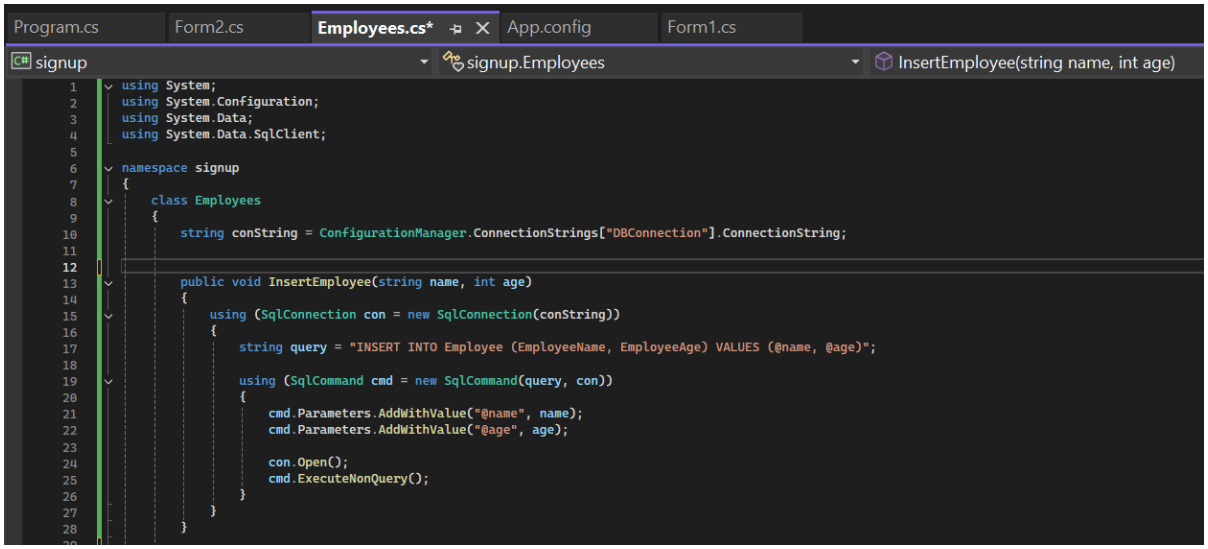


Figure 8.4 — Employees.cs: connection string retrieval and InsertEmployee() method

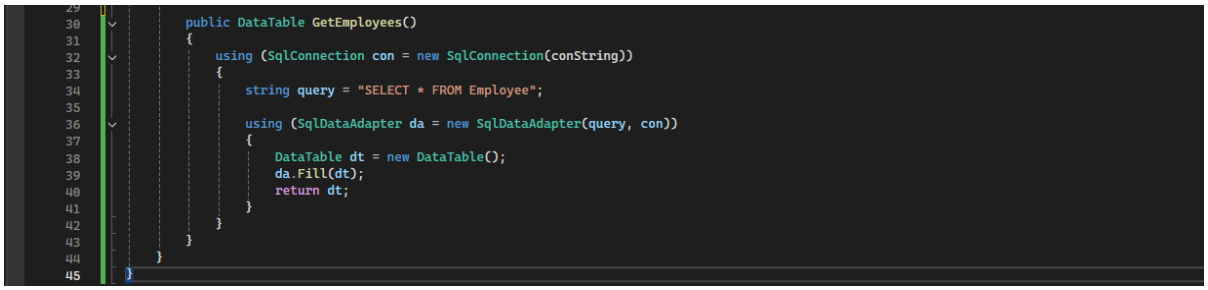


Figure 8.5 — Employees.cs: GetEmployees() method

Form1.cs — Signup / Entry Form

Form1 collects the Name and Age input, validates that neither field is empty and that Age is a valid integer, then opens Form2 (passing the validated values through its constructor).

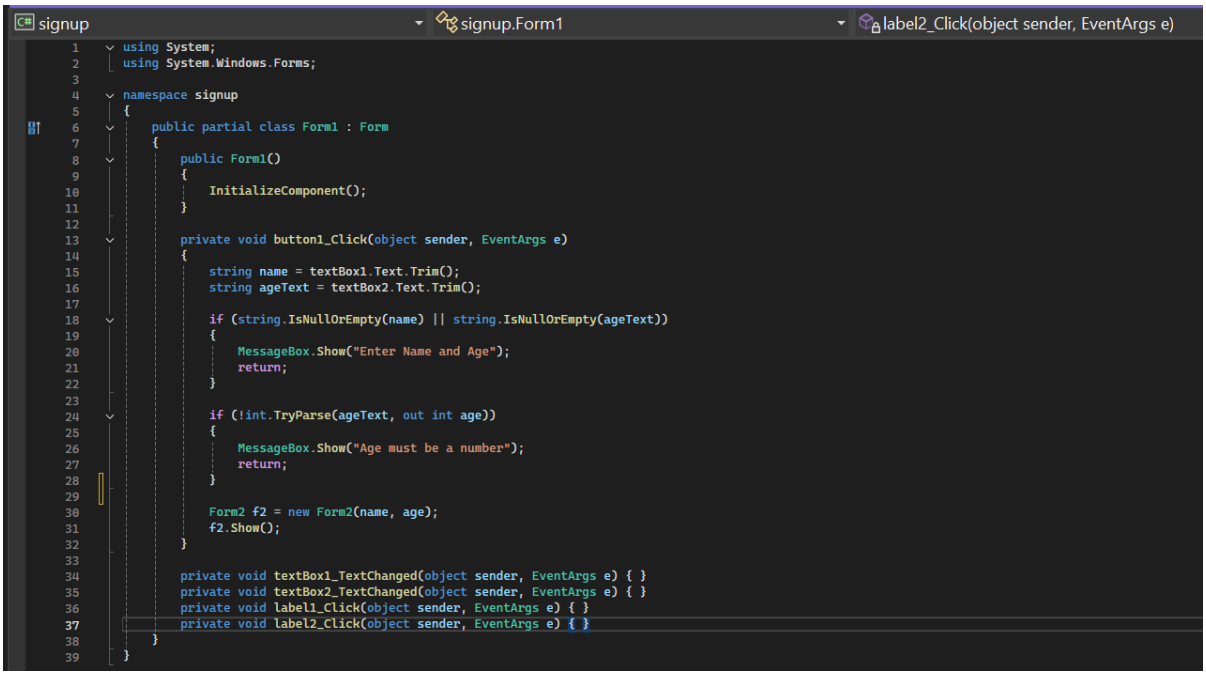


Figure 8.6 — Form1.cs: input validation and navigation to Form2

Form2.cs — Employees List Form

Form2 receives the name and age via its constructor, calls Employees.InsertEmployee() to persist the new record, then calls Employees.GetEmployees() and binds the returned DataTable to the DataGridView, wrapped in a try-catch block to surface any database errors.

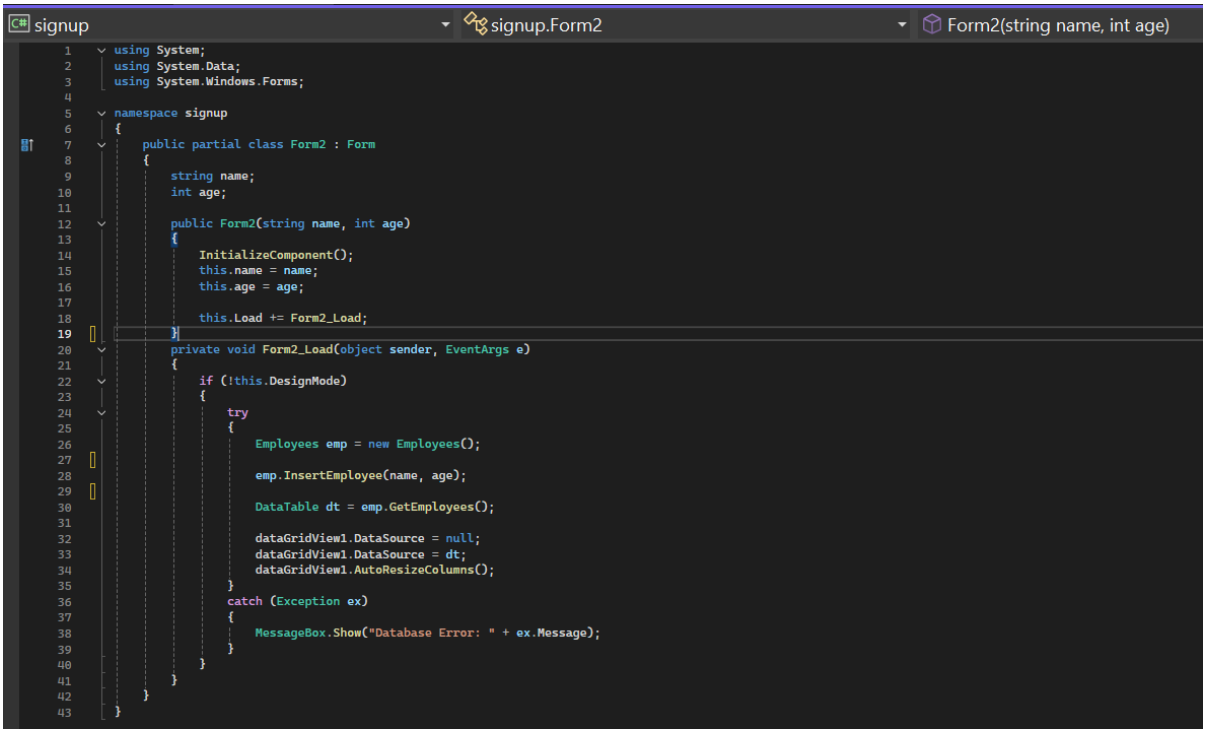


Figure 8.7 — Form2.cs: insert-and-refresh logic bound to the DataGridView

Program.cs — Application Entry Point

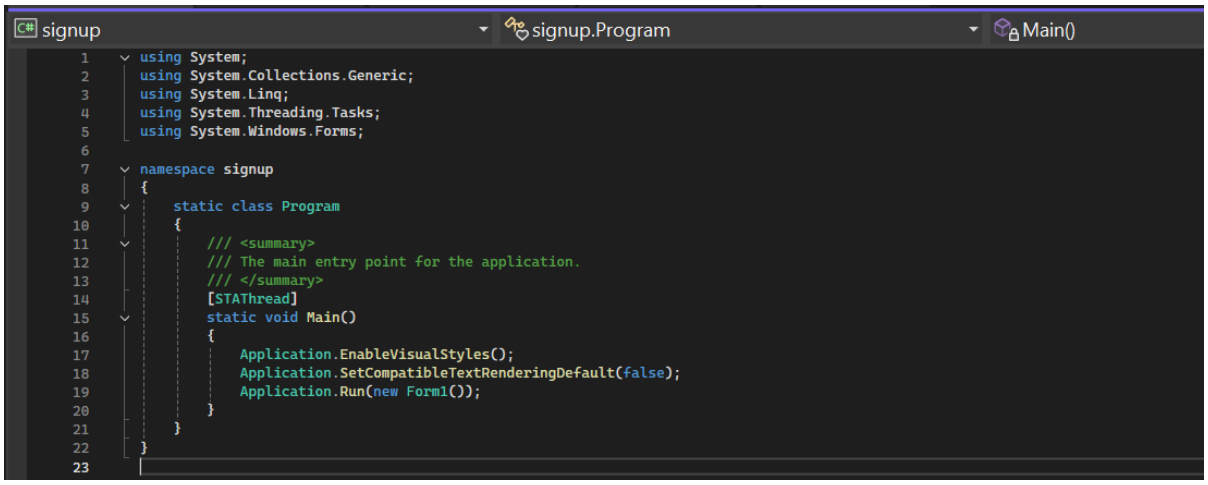


Figure 8.8 — Program.cs: application startup, launching Form1

App.config — Connection String

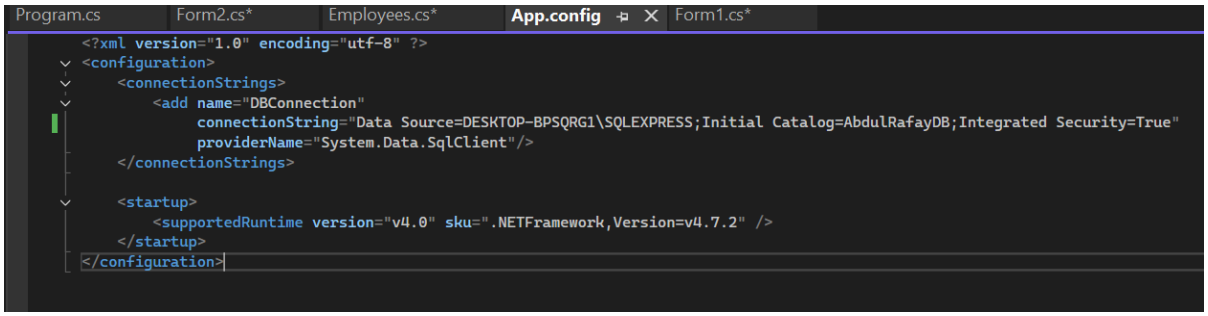


Figure 8.9 — App.config: DBConnection connection string using Integrated Security

Conclusion

This Open Ended Lab consolidates the core skills built across the earlier labs into a small, complete, self-contained application: a form for data entry, a dedicated data access class (DbCon-style) using ADO.NET, parameterized queries for safe insertion, and a DataGridView bound to a live database query. The same pattern — entry form, data-access class, grid-bound list form — is the pattern extended into a full Library Management System across the remaining labs.

Lab 9: Entity Framework — Code First Approach

Objective

To rebuild the data access layer using Entity Framework with the Code First approach, replacing raw ADO.NET data access.

Theory

Entity Framework (EF) is an Object-Relational Mapper (ORM) for .NET that enables developers to work with a database using .NET objects, eliminating most of the manual data-access code (SqlConnection, SqlCommand, etc.) required in ADO.NET.

ORM (Object-Relational Mapping) is a technique that maps object-oriented classes to relational database tables, allowing developers to manipulate data using objects instead of writing SQL directly.

Approaches in Entity Framework

35. **Database First:** EF generates classes from an existing database.
36. **Model First:** The developer designs a visual model, and EF generates both the classes and the database.
37. **Code First:** The developer writes plain C# classes (POCOs), and EF generates the database schema automatically based on those classes. This lab focuses on the Code First approach.

Key Code First Concepts

- **DbContext:** The primary class that manages entity objects at runtime, including querying, saving, and change-tracking. It represents a session with the database.
- **DbSet<T>:** Represents a collection/table for a given entity type, supporting LINQ queries.
- **POCO Entity Classes:** Plain classes representing tables (e.g., Book, Student, Transaction).
- **Data Annotations / Fluent API:** Used to configure entity mapping — e.g., [Key], [Required], [MaxLength(100)], or Fluent API in OnModelCreating() for defining relationships (such as one-to-many between Book and Transaction).
- **Migrations:** A version-control mechanism for the database schema. Add-Migration creates a migration file capturing schema changes; Update-Database applies them — allowing the schema to evolve alongside the code without manually altering tables.
- **LINQ to Entities:** Querying the database using LINQ syntax instead of SQL.
- **Change Tracking & SaveChanges():** EF tracks changes made to entity objects in memory; calling context.SaveChanges() translates all pending changes (inserts/updates/deletes) into SQL commands and commits them.

Advantages of Code First / EF

- Reduces boilerplate ADO.NET code significantly.
- Strongly typed queries — compile-time checking instead of runtime SQL errors.

- Database schema stays in sync with code via Migrations.
- Easier to maintain relationships between entities.

ADO.NET vs Entity Framework

Aspect	**ADO.NET**	**Entity Framework**
Code	More boilerplate	Minimal, object-based
Query	SQL / Stored Procedures	LINQ
Schema Control	Manual	Automatic via Migrations
Performance	Slightly faster (no abstraction)	Slight overhead due to abstraction

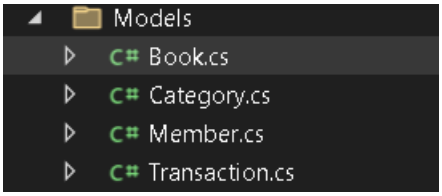
Procedure

38. Install the EntityFramework NuGet package.
39. Define POCO entity classes (Book, Student, Transaction) with relationships.
40. Create a DbContext class exposing DbSet<T> properties.
41. Enable Migrations, run Add-Migration InitialCreate, then Update-Database.
42. Rewrite CRUD operations using LINQ and DbContext instead of raw ADO.NET/Stored Procedures.

Entity Classes Code

```
LibraryAPI                    LibraryAPI.Models.Book          Id
namespace LibraryAPI.Models
{
    17 references
    public class Book
    {
        6 references
        public int Id { get; set; }
        11 references
        public string Title { get; set; } = string.Empty;
        11 references
        public string Author { get; set; } = string.Empty;
        13 references
        public int TotalStock { get; set; }
        17 references
        public int AvailableStock { get; set; }
        8 references
        public int CategoryId { get; set; }
        3 references
        public Category? Category { get; set; }
    }
}
```

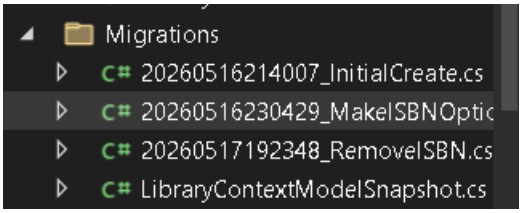
Other models classed



DbContext Code

```
LibraryAPI                    LibraryAPI.Data.LibraryContext          Transactions
using Microsoft.EntityFrameworkCore;
using LibraryAPI.Models;

namespace LibraryAPI.Data
{
    17 references
    public class LibraryContext : DbContext
    {
        0 references
        public LibraryContext(DbContextOptions<LibraryContext> options) : base(options) { }

        4 references
        public DbSet<Category> Categories { get; set; }
        10 references
        public DbSet<Book> Books { get; set; }
        8 references
        public DbSet<Member> Members { get; set; }
        6 references
        public DbSet<Transaction> Transactions { get; set; }
    }
}
```

Migration folder



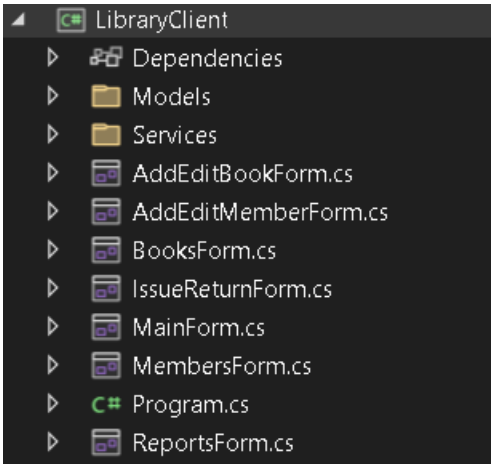
LINQ CRUD Code

```
using System;
using System.Threading.Tasks;
using System.Windows.Forms;
using LibraryClient.Services;
using LibraryClient.Models;

namespace LibraryClient
{
    public partial class BooksForm : Form
    {
        public BooksForm()
        {
            InitializeComponent();

            txtSearch.TextChanged += TxtSearch_TextChanged;
            searchTimer.Tick += SearchTimer_Tick;
            btnAdd.Click += BtnAdd_Click;
            btnEdit.Click += BtnEdit_Click;
            btnDelete.Click += BtnDelete_Click;

            this.Load += BooksForm_Load;
        }

        private async void BooksForm_Load(object? sender, EventArgs e)
        {
            await LoadBooksAsync();
        }

        private async Task LoadBooksAsync(string query = "")
        {
            string endpoint = string.IsNullOrWhiteSpace(query)
                ? "books"
```

```
        : $"books/search?query={Uri.EscapeDataString(query)}";

    var books = await ApiService.GetAsync<Book>(endpoint);
    if (books != null)
    {
        gridBooks.DataSource = books;
        if (gridBooks.Columns["Category"] != null) gridBooks.Columns["Category"].Visible = false;
        if (gridBooks.Columns["CategoryId"] != null) gridBooks.Columns["CategoryId"].Visible = false;
    }
}

private void TxtSearch_TextChanged(object? sender, EventArgs e)
{
    searchTimer.Stop();
    searchTimer.Start();
}

private async void SearchTimer_Tick(object? sender, EventArgs e)
{
    searchTimer.Stop();
    await LoadBooksAsync(txtSearch.Text);
}

private async void BtnAdd_Click(object? sender, EventArgs e)
{
    using (var form = new AddEditBookForm())
    {
        if (form.ShowDialog() == DialogResult.OK)
            await LoadBooksAsync(txtSearch.Text);
    }
}

private async void BtnEdit_Click(object? sender, EventArgs e)
{
    if (gridBooks.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select a book to edit.");
        return;
```

```
        }

        var selectedBook = (Book)gridBooks.SelectedRows[0].DataBoundItem;
        using (var form = new AddEditBookForm(selectedBook))
        {
            if (form.ShowDialog() == DialogResult.OK)
                await LoadBooksAsync(txtSearch.Text);
        }
    }

    private async void BtnDelete_Click(object? sender, EventArgs e)
    {
        if (gridBooks.SelectedRows.Count == 0)
        {
            MessageBox.Show("Please select a book to delete.");
            return;
        }

        var selectedBook = (Book)gridBooks.SelectedRows[0].DataBoundItem;
        var confirm = MessageBox.Show(
            $"Are you sure you want to delete '{selectedBook.Title}'?",
            "Confirm Delete",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Warning);

        if (confirm == DialogResult.Yes)
        {
            bool success = await ApiService.DeleteAsync($"books/{selectedBook.Id}");
            if (success) await LoadBooksAsync(txtSearch.Text);
        }
    }

    private void topPanel_Paint(object sender, PaintEventArgs e)
    {

    }
  }
}
```

Other files structure



Conclusion

Entity Framework Code First simplifies data access considerably, allowing the data layer to be defined and evolved entirely from C# code.

Lab 10: Development using MVC Architecture

Objective

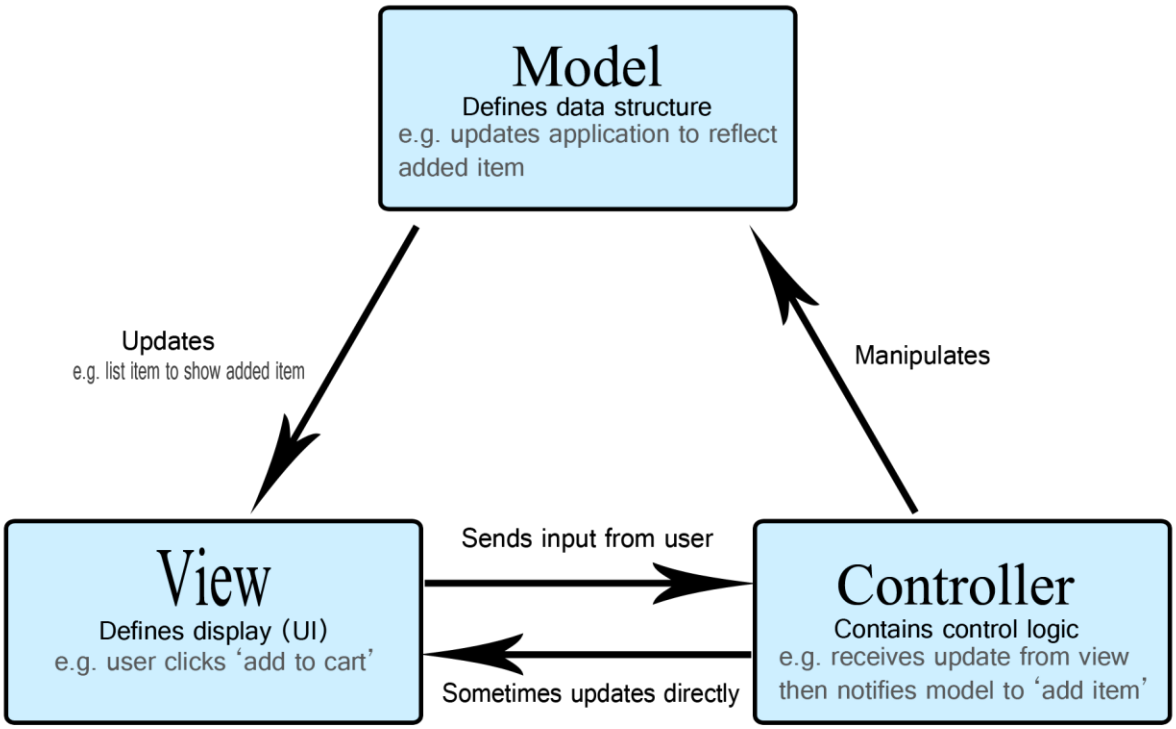
To rebuild the web application using the ASP.NET MVC (Model-View-Controller) architectural pattern, integrating the Entity Framework model built in Lab 9.

Theory

MVC (Model-View-Controller) is a software architectural pattern that separates an application into three interconnected components:

43. **Model:** Represents the application's data and business logic — in this project, the EF entity classes (Book, Student, Transaction) and any supporting ViewModels used to shape data for a view.
44. **View:** Responsible for rendering the UI — in ASP.NET MVC these are .cshtml files using Razor syntax to mix HTML and C#.
45. **Controller:** Handles incoming HTTP requests, interacts with the Model to perform operations, and selects the appropriate View to return. Controllers contain Action Methods mapped to routes.

MVC Request Flow



Routing

ASP.NET MVC uses a routing engine to map URLs to controller actions, typically following the pattern {controller}/{action}/{id} — e.g., /Book/Edit/5 maps to BookController.Edit(int id).

Razor Syntax

A markup syntax that allows embedding C# in HTML using the @ symbol, e.g., iterating over a Model collection to render table rows.

Action Methods

Public methods in a Controller that respond to requests, typically returning an ActionResult (View(), RedirectToAction(), Json(), etc.). They are distinguished by HTTP verb using [HttpGet] and [HttpPost] attributes — commonly a [HttpGet] action displays a form, and a [HttpPost] action processes the submitted data.

Model Binding and ViewModels

MVC automatically maps form data and query strings submitted in a request to method parameters or model objects, reducing manual parsing. A ViewModel is a class specifically designed to carry data between the Controller and the View, often combining data from multiple Models (e.g., a BookIssueViewModel combining Book and Student information for the Issue Book screen) — distinct from the database Model.

Web Forms vs MVC

Aspect	**Web Forms**	**MVC**
State Management	ViewState-heavy	Stateless, lightweight
Control over HTML	Limited (server controls)	Full control
Testability	Difficult	Easy (controllers testable independently)
Separation of Concerns	Weaker	Strong (M-V-C clearly separated)
Learning Curve	Easier for beginners	Requires understanding of HTTP / routing

Procedure

46. Create an ASP.NET MVC project, referencing the EF Code First model from Lab 9.
47. Scaffold or manually create Controllers: BookController, StudentController, TransactionController.
48. Create Razor Views for Index (list), Create, Edit, Delete, and Details for each entity.
49. Implement Issue/Return logic in TransactionController using appropriate ViewModels.

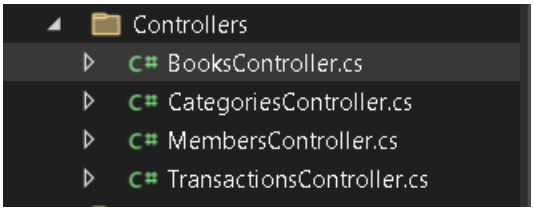
Controller Code

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using LibraryAPI.Data;
using LibraryAPI.Models;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace LibraryAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class BooksController : ControllerBase
    {
        private readonly LibraryContext _context;

        public BooksController(LibraryContext context)
        {
            _context = context;
        }

        // GET: api/books
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Book>>> GetBooks()
        {
            return await _context.Books.Include(b => b.Category).ToListAsync();
        }

        // GET: api/books/5
        [HttpGet("{id}")]
        public async Task<ActionResult<Book>> GetBook(int id)
        {
            var book = await _context.Books.Include(b => b.Category).FirstOrDefaultAsync(b => b.Id == id);

            if (book == null)
            {
                return NotFound();
```

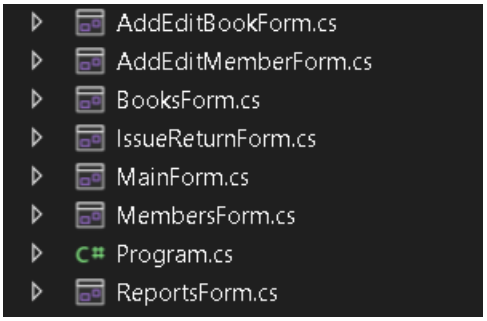
```
    }

    return book;
}

// GET: api/books/search?query=x
[HttpGet("search")]
public async Task<ActionResult<IEnumerable<Book>>> SearchBooks([FromQuery] string query)
{
    if (string.IsNullOrWhiteSpace(query))
    {
        return BadRequest("Search query cannot be empty.");
    }

    return await _context.Books
        .Include(b => b.Category)
        .Where(b => b.Title.Contains(query) || b.Author.Contains(query))
        .ToListAsync();
}

// POST: api/books
[HttpPost]
public async Task<ActionResult<Book>> PostBook(Book book)
{
    // set AvailableStock = TotalStock on creation
    book.AvailableStock = book.TotalStock;

    _context.Books.Add(book);
    await _context.SaveChangesAsync();

    return CreatedAtAction(nameof(GetBook), new { id = book.Id }, book);
}

// PUT: api/books/5
[HttpPut("{id}")]
public async Task<IActionResult> PutBook(int id, Book book)
{
    var existingBook = await _context.Books.FindAsync(id);
```

```
            if (existingBook == null)
            {
                return NotFound();
            }

            // update Title, Author, TotalStock
            existingBook.Title = book.Title;
            existingBook.Author = book.Author;

            // Adjust available stock based on total stock change
            var stockDiff = book.TotalStock - existingBook.TotalStock;
            existingBook.TotalStock = book.TotalStock;
            existingBook.AvailableStock += stockDiff;

            await _context.SaveChangesAsync();

            return NoContent();
        }

        // DELETE: api/books/5
        [HttpDelete("{id}")]
        public async Task<IActionResult> DeleteBook(int id)
        {
            var book = await _context.Books.FindAsync(id);
            if (book == null)
            {
                return NotFound();
            }

            _context.Books.Remove(book);
            await _context.SaveChangesAsync();

            return NoContent();
        }
    }
}
```

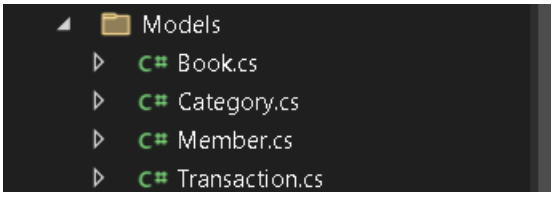
Other files controllers



View folder



Models



Conclusion

The MVC-based version demonstrates a cleaner separation of concerns compared to Web Forms and reflects industry-standard practice for modern ASP.NET web development.

Lab 11: Use of the C# Collection Framework

Objective

To demonstrate the use of C# Collection Framework classes for managing in-memory data within the application, alongside database-backed persistence.

Theory

The **.NET Collection Framework** provides a set of classes and interfaces (in System.Collections and System.Collections.Generic) for storing and manipulating groups of objects in memory, offering more flexibility than plain arrays, which have a fixed size and are not type-flexible for generics.

Non-Generic Collections (System.Collections)

Largely legacy; they store objects as type object, requiring boxing/unboxing for value types:

- ArrayList — a dynamically resizable array
- Hashtable — key-value pairs, unordered
- Stack — LIFO structure
- Queue — FIFO structure

Generic Collections (System.Collections.Generic)

Preferred in modern C# for type safety and performance, since no boxing/unboxing is required.

Collection	Description	Possible Use in this Project
List<T>	Dynamic, ordered, index-accessible array	Holding a temporary list of Book objects fetched from the database before binding to the grid
Dictionary<TKey,TValue>	Key-value pairs with fast lookup	Mapping BookID to a Book object for quick lookup during issue/return
Queue<T>	FIFO — first in, first out	Managing a waitlist / reservation queue for a book that is currently unavailable
Stack<T>	LIFO — last in, first out	Tracking recently viewed books, or an undo history of actions
HashSet<T>	Unordered collection of unique elements	Storing unique ISBNs to quickly check for duplicates before insertion

Collection	Description	Possible Use in this Project
SortedList<TKey,TValue>	Key-value pairs maintained in sorted key order	Displaying books sorted alphabetically by title without a separate database query
LinkedList<T>	Doubly linked list, efficient insertion/removal at both ends	Managing a dynamic borrowing history log

Why Use Collections Alongside a Database?

- **Caching:** Frequently accessed data (e.g., book categories) can be cached in a List<T> or Dictionary<T,T> to avoid repeated database calls.
- **Temporary Processing:** Data fetched from the database is often processed in memory (filtering, sorting, grouping) using collections before being displayed or written back.
- **Business Logic Support:** A reservation Queue<T> is a natural fit for handling 'who is next in line' when a book is unavailable — a scenario not always convenient to model directly in SQL.

LINQ with Collections

Generic collections integrate seamlessly with LINQ (Language Integrated Query), allowing operations such as filtering, sorting, and grouping directly in C# — for example, filtering a list of transactions for overdue items, or sorting a list of books by title.

Complexity Considerations

- List<T> — O(1) access by index, O(n) search
- Dictionary<T,T> — O(1) average lookup by key
- Queue<T> / Stack<T> — O(1) enqueue/dequeue or push/pop

Procedure

50. Implement a Queue<Student> to manage a reservation waitlist for books that are currently out of stock.
51. Use a Dictionary<int, Book> to cache Book objects in memory for fast lookup during the Issue Book process.
52. Use a Stack<string> to maintain a simple 'recently searched books' history.
53. Apply LINQ queries over List<T> collections to generate the Overdue Books report from Lab 8 entirely in memory, as an alternative to a SQL query.

Conclusion

This lab reinforces core C# data-structure concepts and shows how in-memory collections complement database operations for caching, temporary processing, and business-logic scenarios such as reservation

queues — completing the full progression of this project from a simple database connection through to a layered, web-enabled Library Management System.